

Universidad San Carlos de Guatemala

Organización de Lenguajes y Compiladores 2

Ing. Mario Tobar



MANUAL USUARIO

Mario Raúl Yancor Ocaña

201930761

Quetzaltenango, 26 de septiembre de 2026

Uso general de la aplicación

En las siguientes imágenes se muestra el flujo de uso de la aplicación Contacto 3xtrat3rr3str3D. Cada captura incluye una explicación breve de la acción que se realiza y del resultado que debe observar el usuario.



Imagen 1. Pantalla principal de la aplicación

Al iniciar el programa se muestra la ventana principal. En el panel izquierdo se encuentra el árbol del proyecto, en el centro se visualiza el área de trabajo y en la parte inferior aparecen las pestañas de Diagnósticos, Cuartetas, Memoria/C3D, Código C3D y Código C. Desde esta pantalla se inicia el proceso abriendo una carpeta de proyecto para analizar sus archivos fuente.

Opciones principales y validación del análisis

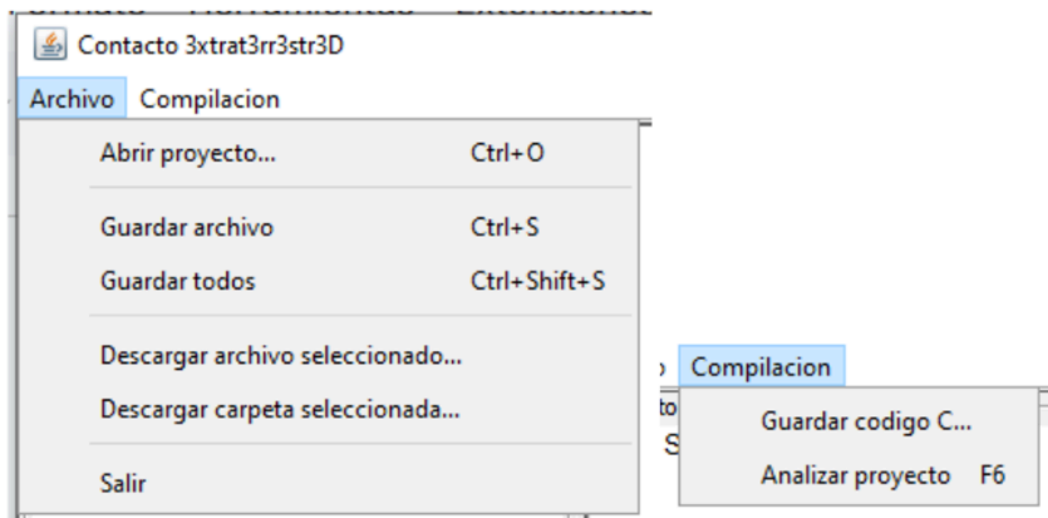


Imagen 2. Menús Archivo y Compilación

En el menú Archivo se encuentran las acciones para abrir un proyecto, guardar el archivo actual, guardar todos los archivos, descargar un archivo seleccionado, descargar una carpeta seleccionada y salir. En el menú Compilación se encuentran las opciones Guardar código C y Analizar proyecto. Para procesar el proyecto se utiliza Analizar proyecto, también disponible mediante la tecla F6.

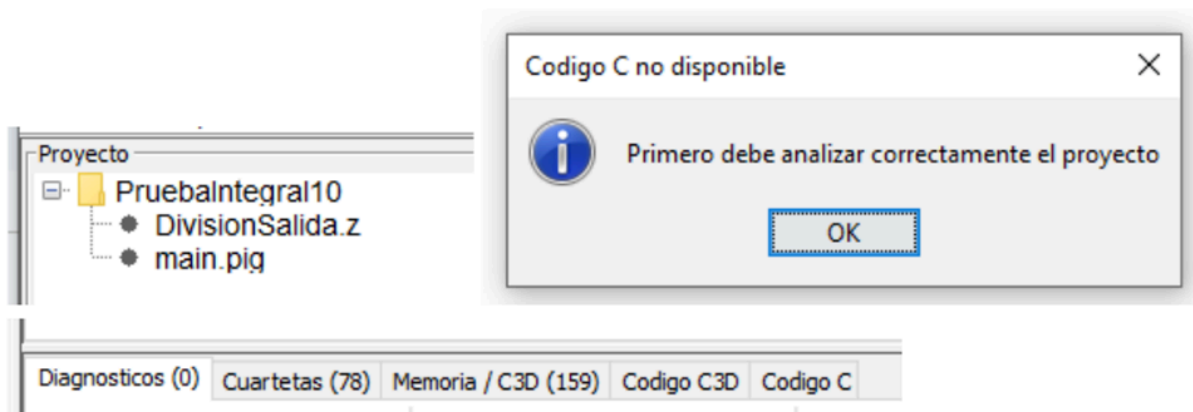


Imagen 3. Validación al intentar obtener el código C

En esta captura ya se observa un proyecto cargado con los archivos DivisionSalida.z y main.pig. Si se intenta obtener o guardar el código C antes de realizar correctamente el análisis, la aplicación muestra el mensaje “Código C no disponible”. Esta validación evita generar un archivo C cuando el proyecto todavía no ha sido analizado de forma satisfactoria.

Resultado del análisis: cuartetas

Diagnostico (0)	Cuartetas (78)	Memoria / C3D (159)	Codigo C3D	Codigo C
#	Operador	Argumento 1	Argumento 2	Resultado
1	INICIO_FUNCION	DivisionSalida.<int>()	DivisionSalida	1
2	DECLARAR_PARAMETRO	DivisionSalida	THIS	THIS
3	RETORNAR	THIS	-	-
4	FIN_FUNCION	DivisionSalida.<int>()	-	-
5	INICIO_FUNCION	DivisionSalida.<diverPositivo>()	int	1
6	DECLARAR_PARAMETRO	DivisionSalida	THIS	THIS
7	MODULO	2	2	r_19
8	RESTAR	2	r_19	r_11
9	DEVOLVER	r_11	2	r_12
10	RETORNAR	r_12	-	-
11	FIN_FUNCION	DivisionSalida.<diverPositivo>()	-	-
12	INICIO_FUNCION	DivisionSalida.<diverCero>()	int	1
13	DECLARAR_PARAMETRO	DivisionSalida	THIS	THIS
14	MODULO	10	3	r_13
15	RESTAR	10	r_13	r_14
16	DEVOLVER	r_14	3	r_15
17	RETORNAR	r_15	-	-
18	FIN_FUNCION	DivisionSalida.<diverCero>()	-	-
19	INICIO_FUNCION	DivisionSalida.<diverNegativo>()	int	1
20	DECLARAR_PARAMETRO	DivisionSalida	THIS	THIS
21	NEGATIVO	2	-	r_16
22	MODULO	r_16	2	r_17
23	RESTAR	r_16	r_17	r_18
24	DEVOLVER	r_18	3	r_19
25	RETORNAR	r_19	-	-
26	FIN_FUNCION	DivisionSalida.<diverNegativo>()	-	-
27	INICIO_FUNCION	DivisionSalida.<diverDecimal>()	double	1
28	DECLARAR_PARAMETRO	DivisionSalida	THIS	THIS
29	DEVOLVER	2.0	2	r_110
30	RETORNAR	r_110	-	-
31	FIN_FUNCION	DivisionSalida.<diverDecimal>()	-	-
32	INICIO_FUNCION	DivisionSalida.<diverAsignacion>()	int	1
33	DECLARAR_PARAMETRO	DivisionSalida	THIS	THIS
34	DECLARAR	x	-	x
35	ASIGNAR	5	-	x
36	MODULO	x	2	r_111
37	RESTAR	x	r_111	r_112
38	DEVOLVER	r_112	2	r_113
39	ASIGNAR	r_113	-	x
40	RETORNAR	x	-	-
41	FIN_FUNCION	DivisionSalida.<diverAsignacion>()	-	-
42	INICIO_FUNCION	DivisionSalida.<proberSalida>()	void	1
43	DECLARAR_PARAMETRO	DivisionSalida	THIS	THIS
44	PARADEAR	x	println(char)	-
45	PARADEAR	5	println(char)	-
46	PARADEAR	10	println(char)	-
47	PARADEAR	true	println(boolean)	-
48	PARADEAR	false	println(boolean)	-
49	RETORNAR	-	-	-
50	FIN_FUNCION	DivisionSalida.<proberSalida>()	-	-
51	INICIO_FUNCION	MAIN	void	0
52	DECLARAR	DivisionSalida	-	prueba
53	CREAR_OBJETO	DivisionSalida.<int>()	0	pig_10
54	ASIGNAR	pig_10	-	prueba
55	ASIGNAR	prueba	0	-
56	LLAMAR	DivisionSalida.<diverPositivo>()	1	pig_11
57	PARADEAR	pig_11	-	-

Imagen 4. Primer bloque de cuartetas generadas

Después de analizar correctamente el proyecto, la pestaña Cuartetas muestra la representación intermedia generada por el compilador. Cada fila contiene una operación y sus operandos, junto con el resultado correspondiente. Esta vista permite revisar cómo las instrucciones del programa fuente fueron traducidas a operaciones intermedias.

Diagnosticos (0) Cuartetas (78)		Memoria / C3D (199)	Codigo C3D	Codigo C
#	Operador	Argumento 1	Argumento 2	Resultado
1	INICIO_FUNCION	DivisorSalida.cinito()	DivisorSalida	1
2	DECLARAR_PARAMETRO	DivisorSalida	THES	THS
3	LEER_STACK	P+1	DivisorSalida	mem_10
4	ESCRIBIR_STACK	P+0	mem_10	-
5	RETORNAR	-	-	-
6	FIN_FUNCION	DivisorSalida.cinito()	-	-
7	INICIO_FUNCION	DivisorSalida.dividirPositivo()	int	1
8	DECLARAR_PARAMETRO	DivisorSalida	THES	THS
9	MODULO	7	2	r_10
10	RESTAR	7	r_10	r_11
11	DIVIDIR	r_11	2	r_12
12	ESCRIBIR_STACK	P+0	r_12	-
13	RETORNAR	-	-	-
14	FIN_FUNCION	DivisorSalida.dividirPositivo()	-	-
15	INICIO_FUNCION	DivisorSalida.dividirCero()	int	1
16	DECLARAR_PARAMETRO	DivisorSalida	THES	THS
17	MODULO	10	3	r_13
18	RESTAR	10	r_13	r_14
19	DIVIDIR	r_14	3	r_15
20	ESCRIBIR_STACK	P+0	r_15	-
21	RETORNAR	-	-	-
22	FIN_FUNCION	DivisorSalida.dividirCero()	-	-
23	INICIO_FUNCION	DivisorSalida.dividirNegativo()	int	1
24	DECLARAR_PARAMETRO	DivisorSalida	THES	THS
25	NEGATIVO	7	-	r_16
26	MODULO	r_16	2	r_17
27	RESTAR	r_16	r_17	r_18
28	DIVIDIR	r_18	2	r_19
29	ESCRIBIR_STACK	P+0	r_19	-
30	RETORNAR	-	-	-
31	FIN_FUNCION	DivisorSalida.dividirNegativo()	-	-
32	INICIO_FUNCION	DivisorSalida.dividirDecimal()	double	1
33	DECLARAR_PARAMETRO	DivisorSalida	THES	THS
34	DIVIDIR	7.0	2	r_110
35	ESCRIBIR_STACK	P+0	r_110	-
36	RETORNAR	-	-	-
37	FIN_FUNCION	DivisorSalida.dividirDecimal()	-	-
38	INICIO_FUNCION	DivisorSalida.dividirAsignacion()	int	1
39	DECLARAR_PARAMETRO	DivisorSalida	THES	THS
40	DECLARAR	int	-	x
41	ESCRIBIR_STACK	P+2	0	-
42	ESCRIBIR_STACK	P+2	9	-
43	LEER_STACK	P+2	int	mem_11
44	MODULO	mem_11	2	r_111
45	LEER_STACK	P+2	int	mem_12
46	RESTAR	mem_12	r_111	r_112
47	DIVIDIR	r_112	2	r_113
48	ESCRIBIR_STACK	P+2	r_113	-
49	LEER_STACK	P+2	int	mem_13
50	ESCRIBIR_STACK	P+0	mem_13	-
51	RETORNAR	-	-	-
52	FIN_FUNCION	DivisorSalida.dividirAsignacion()	-	-
53	INICIO_FUNCION	DivisorSalida.probarSelec()	void	1
54	DECLARAR_PARAMETRO	DivisorSalida	THES	THS
55	PRINTFER	%f	printfchar	-
56	PRINTFER	%f	printfchar	-
57	PRINTFER	%f	printfchar	-

Box: 0 | Cuartetas: 78 | Memoria: 199

Imagen 5. Continuación del listado de cuartetas

La segunda captura muestra la continuación del listado de cuartetas mediante el desplazamiento dentro de la tabla. Con esto se puede revisar el conjunto completo de operaciones generadas, incluyendo asignaciones, llamadas a funciones, operaciones aritméticas y manejo de valores temporales.

Código de tres direcciones generado

Diagnosticos (0)	Cuartetas (78)	Memoria / C3D (159)	Código C3D	Código C
<pre>P = 0; H = 0; func DivisionSalida.<init>(): paramdecl DivisionSalida this; mem_t0 = Stack[(int) (P+1)]; Stack[(int) (P+0)] = mem_t0; return; endfunc DivisionSalida.<init>() func DivisionSalida.dividirPositivo(): paramdecl DivisionSalida this; z_t0 = 7 % 2; z_t1 = 7 - z_t0; z_t2 = z_t1 / 2; Stack[(int) (P+0)] = z_t2; return; endfunc DivisionSalida.dividirPositivo() func DivisionSalida.dividirDiez(): paramdecl DivisionSalida this; z_t3 = 10 % 3; z_t4 = 10 - z_t3; z_t5 = z_t4 / 3; Stack[(int) (P+0)] = z_t5; return; endfunc DivisionSalida.dividirDiez() func DivisionSalida.dividirNegativo(): paramdecl DivisionSalida this; z_t6 = -7; z_t7 = z_t6 % 2; z_t8 = z_t6 - z_t7; z_t9 = z_t8 / 2; Stack[(int) (P+0)] = z_t9; return; endfunc DivisionSalida.dividirNegativo() func DivisionSalida.dividirDecimal(): paramdecl DivisionSalida this; z_t10 = 7.0 / 2; Stack[(int) (P+0)] = z_t10; return; endfunc DivisionSalida.dividirDecimal() func DivisionSalida.dividirAsignacion(): paramdecl DivisionSalida this; decl int x;</pre>				

ticos: 0 | Cuartetas: 78 | Memoria: 159

Imagen 6. Pestaña Código C3D

En la pestaña Código C3D se visualiza el código de tres direcciones producido a partir de las cuartetas. Se observan variables temporales, llamadas a funciones, parámetros, operaciones aritméticas y el uso de Stack para guardar o recuperar valores. Esta representación permite comprobar la traducción del programa antes de generar el código C final.

Código C generado

```
Diagnosticos (0) Cuartetas (78) Memoria / C3D (159) Código C3D Código C

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <stdbool.h>
#include <string.h>

#define STACK_SIZE 100000
#define HEAP_SIZE 1000000

double Stack[STACK_SIZE];
double Heap[HEAP_SIZE];
int P = 0;
int H = 0;

static void runtime_print_string(double referencia, int salto) {
    int posicion = (int) referencia;
    while (posicion >= 0 && posicion < HEAP_SIZE && Heap[posicion] != -1) {
        putchar((unsigned char) ((int) Heap[posicion]));
        posicion++;
    }
    if (salto) {
        putchar('\n');
    }
}

static void runtime_append_string(double referencia) {
    int posicion = (int) referencia;
    if (posicion < 0) {
        const char *texto = "null";
        for (int i = 0; texto[i] != '\0'; i++) {
            if (H < HEAP_SIZE) Heap[H++] = (unsigned char) texto[i];
        }
        return;
    }
    while (posicion >= 0 && posicion < HEAP_SIZE && Heap[posicion] != -1) {
        if (H < HEAP_SIZE) Heap[H++] = Heap[posicion];
        posicion++;
    }
}

static void runtime_append_number(double valor) {
    char buffer[128];
    int cantidad = snprintf(buffer, sizeof(buffer), "%.15g", valor);
    if (cantidad < 0) return;
    for (int i = 0; i < cantidad && i < (int) sizeof(buffer); i++) {
        if (H < HEAP_SIZE) Heap[H++] = (unsigned char) buffer[i];
    }
}

<
```

sticos: 0 | Cuartetas: 78 | Memoria: 159

Imagen 7. Pestaña Código C

Una vez completado el análisis, la pestaña Código C presenta el código fuente C generado automáticamente. En la parte inicial se incluyen las bibliotecas necesarias, la definición de Stack y Heap y las funciones auxiliares del entorno de ejecución. Este código es el resultado final de la traducción y posteriormente puede guardarse como un archivo con extensión .c para compilarlo con GCC.

Descarga de archivos y carpetas del proyecto

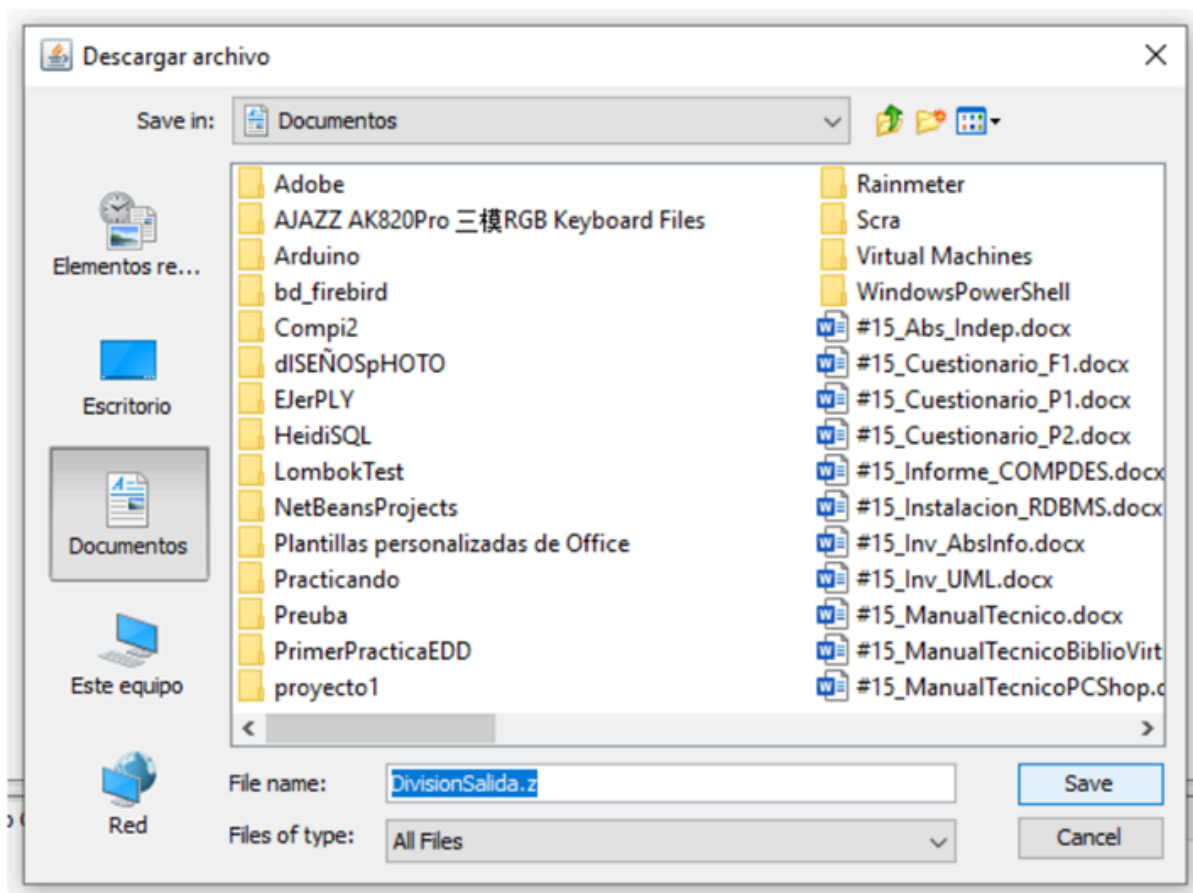


Imagen 8. Descarga de un archivo seleccionado

La opción Descargar archivo seleccionado permite guardar una copia del archivo que se encuentre seleccionado en el árbol del proyecto. En la imagen se está guardando DivisionSalida.z, por lo que el usuario elige la ubicación de destino y confirma la descarga con el botón Save.

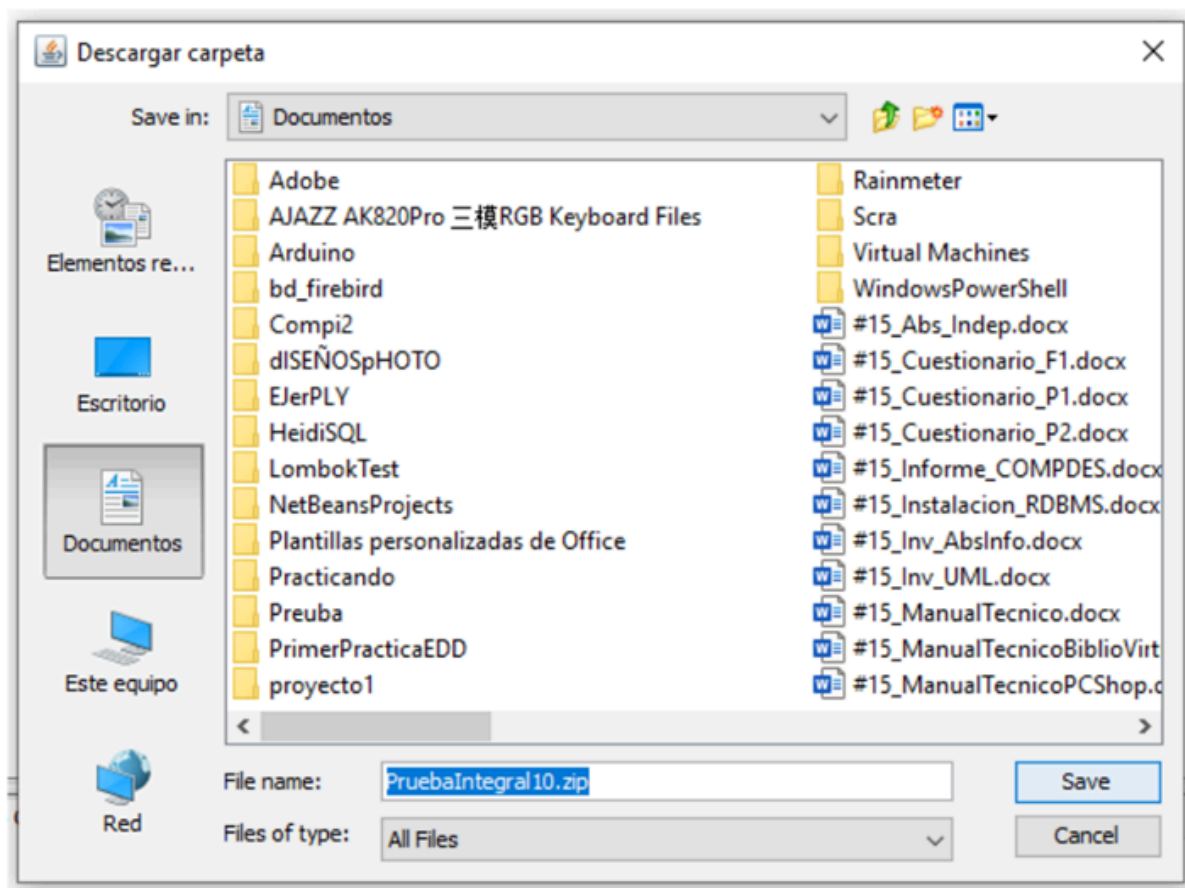


Imagen 9. Descarga de la carpeta completa

La opción Descargar carpeta seleccionada permite exportar el proyecto completo. La aplicación prepara la carpeta seleccionada como un archivo comprimido ZIP; en el ejemplo se guarda como PruebaIntegral10.zip. Esto facilita trasladar o respaldar todos los archivos del proyecto en una sola descarga.

Guardado, compilación y ejecución del código generado

Este equipo > Descargas > PruebaIntegral10 > PruebaIntegral10				
Nombre	Fecha de modificación	Tipo	Tamaño	
DivisionSalida.z	25/09/2026 1:05	Archivo WinRAR	1 KB	
LEEME.txt	23/09/2026 8:13	Documento de te...	1 KB	
main.pig	23/09/2026 8:13	Archivo PIG	1 KB	
PruebaIntegral10.c	23/09/2026 2:16	C Source File	9 KB	
PruebaIntegral10.exe	23/09/2026 2:16	Aplicación	96 KB	
PruebaSinLM.exe	24/09/2026 2:47	Aplicación	96 KB	
SALIDA_ESPERADA.txt	23/09/2026 8:13	Documento de te...	1 KB	

Imagen 10. Archivos utilizados para la prueba integral

En el explorador de archivos se observa la carpeta de prueba con los archivos fuente del proyecto, archivos de apoyo y ejecutables. También aparece el archivo PruebaIntegral10.c, que corresponde al código C generado por la aplicación. Esta carpeta sirve para realizar la comprobación final fuera de la interfaz.

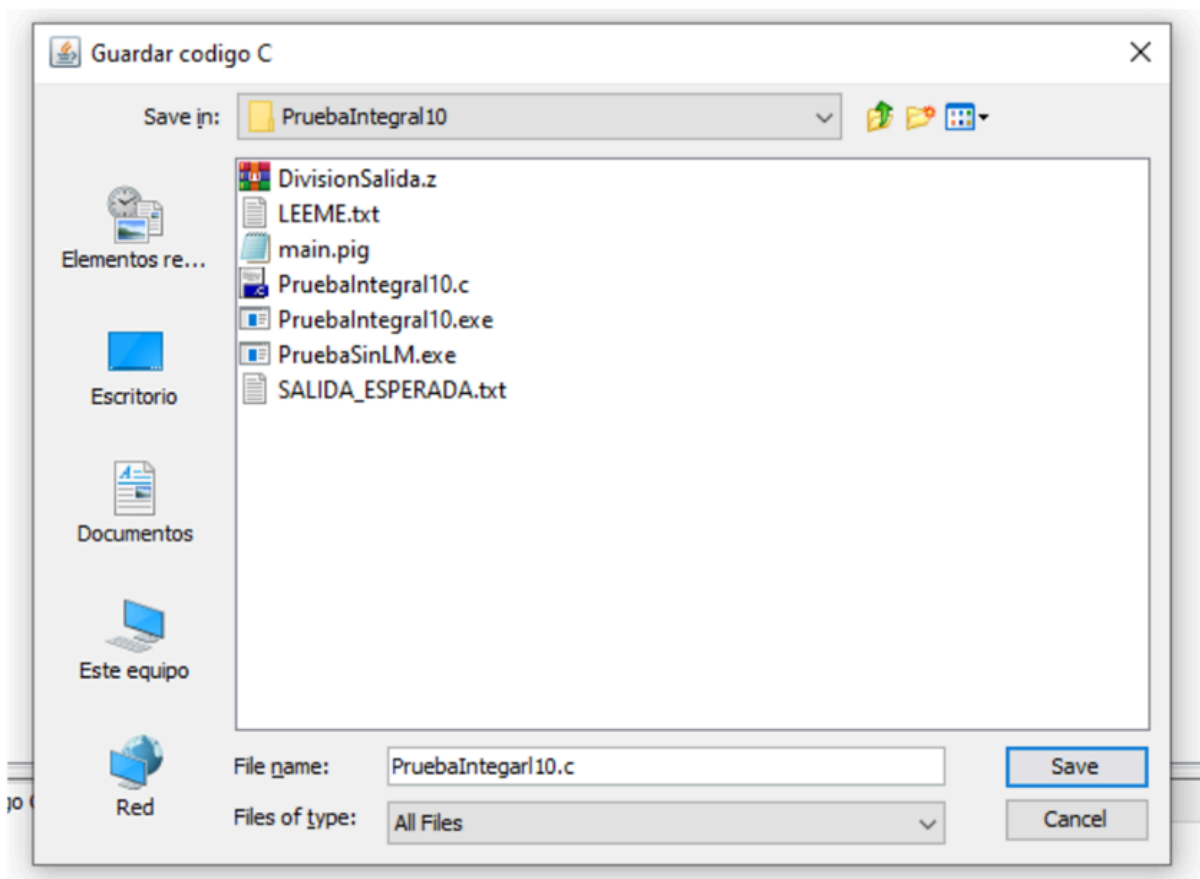


Imagen 11. Guardado del código C

Al seleccionar Guardar código C, se abre el cuadro de diálogo para elegir la ubicación y el nombre del archivo. En el ejemplo el código generado se guarda como PruebaIntegral10.c dentro de la carpeta de la prueba integral. Este archivo es el que posteriormente se compila con un compilador de C.

Comprobación de la salida

```
PS C:\Users\mario\Downloads\PruebaIntegral10\PruebaIntegral10> gcc PruebaIntegral10.c -o PruebaSinLM.exe
PS C:\Users\mario\Downloads\PruebaIntegral10\PruebaIntegral10> .\PruebaSinLM.exe
3
3
-3
3.5
4
A
BC
true
false
```

Imagen 12. Compilación y ejecución desde PowerShell

Finalmente se abre PowerShell en la carpeta del proyecto. Primero se compila el archivo C generado mediante el comando `gcc PruebaIntegral10.c -o PruebaSinLM.exe`. Después se ejecuta `.\PruebaSinLM.exe`. La salida mostrada en consola permite comprobar que el código generado puede compilarse y ejecutarse correctamente, y que produce los resultados esperados para la prueba realizada.

Resultado del flujo: el usuario abre el proyecto, lo analiza, revisa las representaciones intermedias, genera el código C, lo guarda y finalmente lo compila y ejecuta para validar el funcionamiento del programa traducido.